Digital System Design with Verilog

Course 03603473-65

BOARD VERIFICATION SIGN-OFF

Student has programmed the DE1, verified the BCD Adder on hardware, recorded results in the table above, and completed the reflection questions.

Instructor signature: _____ Date: _____ Marks awarded: _____ / 10

Lab 1 — Verilog Fundamentals & First FPGA Design

Name: _____	Student ID: _____

Learning Objectives	By the end of this lab, you will be able to: **(1)** declare a Verilog module with correct port and data-type syntax, **(2)** implement combinational logic using wire, assign, and bitwise operators, **(3)** identify and fix common Verilog syntax errors, and **(4)** follow an instructor-led FPGA demo and record hardware observations.

Part A · Module Declaration & Data Types (Exercises 1–2)

FILL-IN-BLANK	Exercise 1 — Half Adder Module (Complete the Blanks)	5 pts/_____

How a Half Adder behaves:

Inputs: Two 1-bit signals — call them a and b (each can be 0 or 1)

Outputs: sum and carry

sum = a **XOR** b — outputs 1 when the inputs are **different**, 0 when they are the same

carry = a **AND** b — outputs 1 only when **both** inputs are 1 (i.e. 1+1 generates a carry of 1)

Think of it as 1-bit addition: 1+1 = 10 in binary → sum=0, carry=1.
Any other combination gives carry=0.

a	b	sum (a XOR b)	carry (a AND b)	Interpretation
0	0	0	0	0 + 0 = 0, no carry
0	1	1	0	0 + 1 = 1, no carry
1	0	1	0	1 + 0 = 1, no carry
1	1	0	1	$1 + 1 = 10_2$ → sum=0, carry=1

Complete the code below. A **Half Adder** takes two 1-bit inputs and outputs **sum** (XOR) and **carry** (AND). Fill in every ___ — there are **5 blanks** to fill.

```
module _____ (          // 1. module name
  input  wire _____ ,   // 2. first input signal name
  input  wire _____ ,   // 3. second input signal name
```

```
  output wire sum,
  output wire carry
);

  assign sum   = _____________________ ^ _____________________ ; // 4.  use same names as blanks 2 & 3
  assign carry = _____________________ & _____________________ ; // 5.  use same names as blanks 2 & 3

endmodule
```

DEBUG	Exercise 2 — Find 3 Bugs in This Full Adder Code	6 pts/_____

How a Full Adder behaves:
Inputs: a, b (1-bit operands) + cin (carry-in from a previous stage, also 1-bit)
Outputs: sum (the result bit) + carry_out (carry passed to the next stage)

sum = a XOR b XOR cin
carry_out = (a AND b) OR (cin AND (a XOR b))

A Full Adder extends the Half Adder by accepting a carry-in, making it chainable — the carry_out of one stage feeds into the cin of the next.

a	b	cin	sum	carry_out	Interpretation
0	0	0	0	0	0+0+0 = 0
0	1	1	0	1	0+1+1 = 2 = 10_2 → sum=0, carry=1
1	0	1	0	1	1+0+1 = 2 = 10_2 → sum=0, carry=1
1	1	1	1	1	1+1+1 = 3 = 11_2 → sum=1, carry=1

The module below *should* implement a Full Adder but it has **exactly 3 bugs**. Circle or underline each bug in the listing, then fill in the table.

```
module full_adder (
  input a, b, cin,
  output sum, carry_out
);

  wire p, g;
  assign p         = a ^ b;
  assign g         = a & b;
  assign sum       = p & cin;
  assign carry_out= g | (p & cin);

EndModule
```

Bug #	Line / Location	What is wrong?	Corrected code
1			
2			
3			

Part B · Operators (Exercise 3)

MATCH	Exercise 3 — Match the Operator to Its Function	5 pts/_____

Write the matching letter in the Answer column.

Operator	Answer	Function	Letter
^	_____	Logical AND — returns 1 single bit: true or false	A
&	_____	Bitwise XOR — operates on each bit independently	B
&&	_____	Bitwise AND — operates on each bit independently	C
|	_____	Concatenation — joins bits into a wider vector	D
{ }	_____	Bitwise OR — operates on each bit independently	E

Part C · Module Specification (Exercise 4)

DESIGN	Exercise 4 — Write a Full Adder from Specification	8 pts/_____

Using the specification below, write the complete Full Adder module. Use assign statements only — no structural instantiation.

Port	Direction	Width	Boolean Expression
a	input	1 bit	(operand A)
b	input	1 bit	(operand B)
cin	input	1 bit	(carry-in)
sum	output	1 bit	sum = a XOR b XOR cin
cout	output	1 bit	cout = (a AND b) OR (cin AND (a XOR b))

Part D · Integrated FPGA Project (Exercises 5–7) — Real Circuit, Real Board

Mission Brief — BCD Adder with 7-Segment Display (DE1)

Exercises 5 and 6 form one integrated project. You will complete a **BCD Adder** that adds two decimal digits (each 0–9) and displays the result (0–18) on the DE1 board's 7-segment displays. **Program the board and demonstrate to your instructor.** | **Inputs:** SW[3:0] = A · SW[7:4] = B **Outputs:** HEX0 = units digit · HEX1 = tens digit

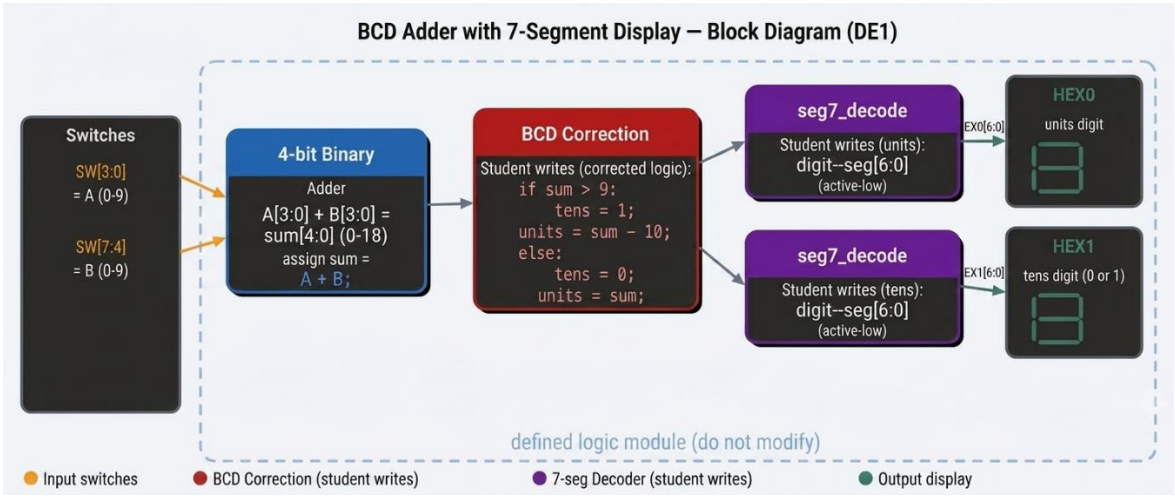


Figure — BCD Adder Block Diagram. The de1_top wrapper (dashed box) is given.
You write: (1) BCD correction logic and (2) the 7-segment decoder.
Both use only assign — no always blocks needed.

de1_top.v — Given to you. Do NOT modify. Add this file to your Quartus project along with your bcd_correct.v and seg7_decode.v.

```
module de1_top (input [9:0] SW,
  output [6:0] HEX0, HEX1, HEX2, HEX3);

  wire [4:0] sum;
  wire [3:0] units, tens;

  binary_adder u_add (.a(SW[3:0]), .b(SW[7:4]), .sum(sum));
  bcd_correct  u_bcd (.sum(sum),   .units(units), .tens(tens));
  seg7_decode  u_hex0(.digit(units),.seg(HEX0));
  seg7_decode  u_hex1(.digit(tens), .seg(HEX1));
  assign HEX2 = 7'h7F; assign HEX3 = 7'h7F;
endmodule
```

COMPLETE + FPGA	Exercise 5 — BCD Adder with 7-Segment Display	15 pts/_____

5A — Write the binary_adder Module from Specification

Module Specification: binary_adder
Purpose: Add two 4-bit unsigned integers and produce a 5-bit result.
Why 5 bits? Maximum input is 9+9 = 18. In binary: 18 = 10010_2 — this requires 5 bits (bit positions 4 down to 0).

Port	Direction	Width	Description	Example value
a	input	[3:0] 4 bits	Operand A — one BCD digit, constrained to 0-9	SW[3:0] = 4'b0111 → A=7
b	input	[3:0] 4 bits	Operand B — one BCD digit, constrained to 0-9	SW[7:4] = 4'b0110 → B=6
sum	output	[4:0] 5 bits	Binary sum A+B. Range: 0 to 18. MSB (bit 4) acts as the carry-out.	7+6=13 = 5'b01101

Behaviour & design rules:

1. **Combinational only** — output changes immediately when a or b changes. Use assign, not always.
2. **Bit-width matters** — in Verilog, 4'b1001 + 4'b1001 = 4'b0010 (overflow!). Pad both inputs to 5 bits first: {1'b0, a}.
3. **Worked example**: A=5, B=8 → {0,0101} + {0,1000} = 01101 → sum = 5'b01101 = 13 ✓

Write your complete binary_adder module in the space below:

```
// binary_adder.v — write your code here
```

5B — BCD Correction Logic (fill in 3 blanks)

When A + B > 9, the tens digit becomes 1 and the units digit = sum − 10. Complete the bcd_correct module below using only assign statements.

```
// bcd_correct.v  — given skeleton, fill in the blanks
module bcd_correct (
  input  [4:0] sum,     // binary sum 0-18 from the adder
  output [3:0] units,   // units digit  0-9
  output [3:0] tens     // tens digit   0 or 1
);

  // tens digit: 1 only when sum >= 10
  assign tens  = ( sum >= ______ ) ? 4'd1 : 4'd0;

  // units digit: sum - 10 when sum >= 10, else sum itself
  assign units = ( sum >= ______ ) ? (sum - ______) : sum[3:0];
  //                                  ^ what do we subtract?
endmodule

// Hint: the threshold is 10, and you subtract 10 to get the units digit
```

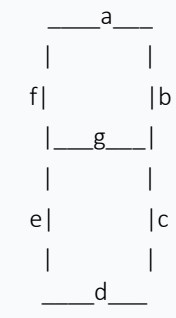
Note: Both blanks in this module are related — the same threshold value appears twice, and the subtracted value equals that threshold.

Hint *9 + 1 = 10. The tens digit first becomes 1 exactly when sum reaches 10. To get units from 10→0, 11→1, 12→2 ... subtract 10.*

5C —7-Segment Decoder (fill in 4 blanks)

DE1 7-segment segments are **active-low** (0 = on, 1 = off). Complete the decoder for the **4 missing rows** in the case statement (digits 4, 5, 6, 7).

Segment layout (active-low):



HEX[6:0] = {g,f,e,d,c,b,a}

Segment truth table:

Digit	gfedcba	HEX[6:0]
0	0111111	7'b011_1111
1	0000110	7'b000_0110
2	1011011	7'b101_1011
3	1001111	7'b100_1111
4	....	
5	....	
6	....	
7	....	
8	1111111	7'b111_1111
9	1101111	7'b110_1111

```
// seg7_decode.v  — complete the 4 missing case entries
module seg7_decode (
  input  [3:0] digit,
  output reg [6:0] seg   // active-low: {g,f,e,d,c,b,a}
);
  always @(*) begin
```

```
  case (digit)
    4'd0: seg = 7'b011_1111;  // 0
    4'd1: seg = 7'b000_0110;  // 1
    4'd2: seg = 7'b101_1011;  // 2
    4'd3: seg = 7'b100_1111;  // 3
    4'd4: seg = 7'b_____________________; // 4  ← fill in
    4'd5: seg = 7'b_____________________; // 5  ← fill in
    4'd6: seg = 7'b_____________________; // 6  ← fill in
    4'd7: seg = 7'b_____________________; // 7  ← fill in
    4'd8: seg = 7'b111_1111;  // 8
    4'd9: seg = 7'b110_1111;  // 9
    default: seg = 7'b000_0000;
  endcase
 end
endmodule
```

PROGRAM + VERIFY	Exercise 6 — Program DE1 & Verify on Board	10 pts/_____

Your task: Ask your instructor to compile and program the DE1 board with your completed code. Then test your circuit with the switch combinations below and record what the 7-segment displays show.

Verification Table — set the switches, observe the 7-segment display, and record what you see:

Inputs (A + B)	Observed on DE1
A=0, B=0 (SW=0000 0000)	HEX1=_____, HEX0=_____
A=3, B=4 (SW=0100 0011)	HEX1=_____, HEX0=_____
A=5, B=5 (SW=0101 0101)	HEX1=_____, HEX0=_____
A=7, B=6 (SW=0110 0111)	HEX1=_____, HEX0=_____
A=9, B=9 (SW=1001 1001)	HEX1=_____, HEX0=_____
A=2, B=8 (SW=1000 0010)	HEX1=_____, HEX0=_____
A=6, B=3 (SW=0011 0110)	HEX1=_____, HEX0=_____
A=9, B=1 (SW=0001 1001)	HEX1=_____, HEX0=_____